

CANable V2.5 Candlelight&Slcan 更新说明

目录

CANable V2.5 Candlelight&Slcan 更新说明	1
第 1 部分 概述.....	3
1.1 前言	3
1.2 关于 Slcan 和 Candlelight	3
1.3 关于 HUD ECU Hacker	3
第 2 部分 说明手册	4
2.1 CANable 硬件介绍	4
2.2 原版硬件设计缺陷	4
2.3 CANable 官方固件	5
2.4 Slcan 协议	5
2.5 Candlelight 协议	5
2.6 USB 数据传输效率	6
2.7 Candlelight 固件与 Windows 兼容性问题	6
2.8 全新固件烧录程序	7
2.9 LED 指示灯含义	8
2.10 上位机软件	8
2.11 CANbus 设置	9
2.12 CAN 总线过滤器	9
2.13 采样点和波特率	9
2.14 CANbus 错误	11
2.15 发送超时	12
2.16 发射回波标记	13
2.17 时间戳	13
2.18 DFU 模式	13
2.19 Bug 修复	14
2.20 总线负载	14
2.21 收发器延迟	14
第 3 部分 Slcan 应用程序开发手册	15

3.1 Sscan 指令	15
3.2 Sscan 响应和事件	20
3.3 Sscan 版本信息	21
3.4 Sscan 数据包	23
3.5 Sscan 错误报告	23
3.6 Sscan 初始化	24
第 4 部分 Candlelight 应用程序开发手册	26
4.1 C++ 演示应用程序	26
4.2 新的 ElmüSoft 协议	27
4.3 进入 DFU 模式	27
第 5 部分 固件开发人员手册	28
5.1 注意事项	28
5.2 添加新开发板	28
5.3 添加新处理器	28
5.4 USB 接口	28
5.5 在 Windows 上编译 STM32 代码	28
第 6 部分 版本说明	30
6.1 源代码版本	30
6.2 更新提示	30

第 1 部分 概述

1.1 前言

CANable 2.5 固件是在 CANable 官方固件 Candlelight 和 Slcan 的基础上修改，为 CANable V2.0 更新的一个开源固件，作者是 Elmue。

官网下载到的固件存在着诸多问题且很长时间未被解决，全新的 CANable 2.5 固件不仅修复了一系列的 bug，改善了与现在主流 Windows 系统版本的兼容性，还增加了许多功能，让 CANable 更适用于专业的 CAN 总线分析和应用。

不仅如此，Elmue 还把 CANable 适配器融入到了其开发的上位机软件 HUD ECU Hacker 中，使用其中的固件更新程序，可以将固件一键上传到 CANable 设备。

本篇文章将着重介绍固件更新的内容，有关固件烧录和 HUD ECU Hacker 的安装使用，请参考我们的另外一篇教程。

Elmue 对旧版 CANable 固件做了很多吐槽，很有意思，我们也十分推荐您阅读原文。

原文：https://netcult.ch/elmue/CANable%20Firmware%20Update/#Sample_Baud

HUD ECU Hacker: <https://netcult.ch/elmue/HUD%20ECU%20Hacker/>

GitHub: <https://github.com/Elmue/CANable-2.5-firmware-Slcan-and-Candlelight>

1.2 关于 Slcan 和 Candlelight

- 1、两个原本独立的固件项目 Slcan 和 Candlelight 被合并成一个项目，并使用了相同的代码库；
- 2、全新的 CANable 2.5 固件质量优秀，并且是开源的；
- 3、在使用旧版固件时，如果 CAN 通信无法正常工作，会很难找到原因，CANable 2.5 固件增加了报错机制，方便您找到问题所在；
- 4、尽管增加了很多新功能，但新版本固件与旧版本上位机仍然兼容；
- 5、该固件已成功测试，支持高达 10 兆波特的 CAN FD 波特率；
- 6、该固件的设计使其易于扩展，以满足未来的需求；
- 7、该固件由一位拥有 40 多年编程、逆向工程和电子技术经验的软件开发人员编写。

1.3 关于 HUD ECU Hacker

- 1、使用固件更新程序，可以一键将新固件上传到 CANable 设备。
- 2、附带一个高速 CAN 原始终端，它不仅支持所有新功能，还能与旧式适配器兼容。
- 3、可以获取 CAN 总线流量的日志文件。
- 4、可以解码摩托车、汽车、卡车和船舶中使用的 ISO 15765、J1939 和 NMEA 2000 CAN 总线协议。
- 5、可以提供 HUD ECU Hacker 用于解码 CAN 总线流量的 DBC 文件。
- 6、可以编写与 CAN 总线设备通信的超高速宏脚本。

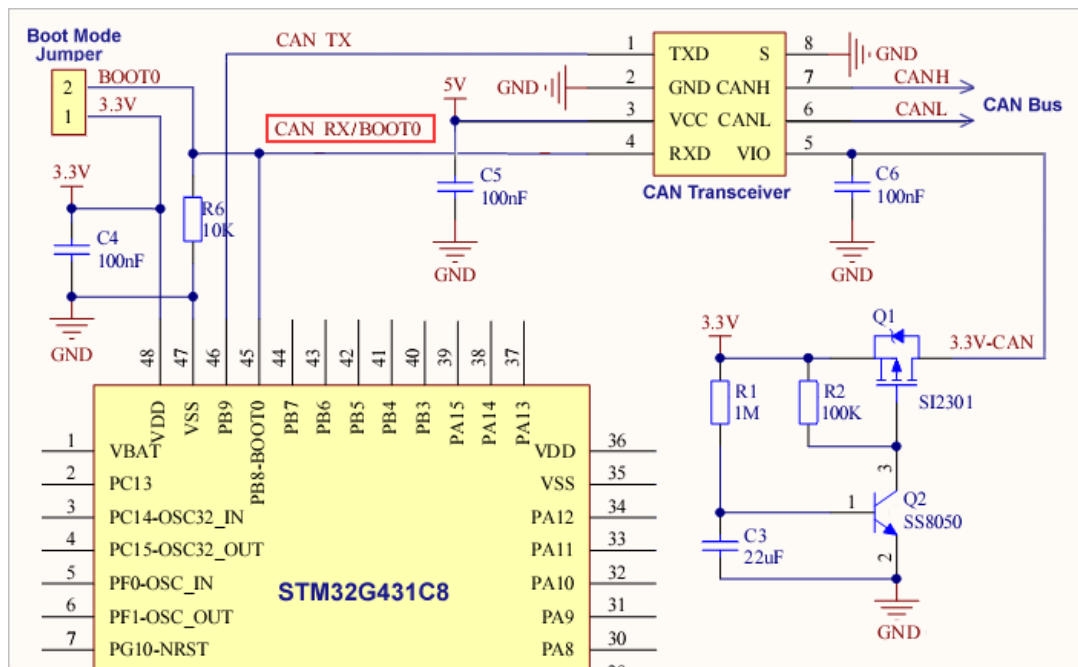
第 2 部分 说明手册

2.1 CANable 硬件介绍

CANable 是一款物美价廉的 CAN 总线适配器，支持 CAN FD 协议。如果您要使用 2.5 固件，请确保它是 CANable 2.0 版本，并且采用 STM32G431 处理器。Elmue 使用的是 MKS CANable 2.0 Pro S，这是一款带外壳与电气隔离的 CANable 2.0 适配器，采用了 ADM3050E 隔离式 CAN 收发器和 B05050S-1WR3 隔离式 DC-DC 转换器。它还带有一个手动开关，可以将一个 $120\ \Omega$ 的终端电阻连接到 CAN 总线上。Elmue 对其进行了全面测试，在 10 兆波特率下也能完美运行。

2.2 原版硬件设计缺陷

CANable 官方硬件设计存在一个缺陷，那就是使用了同一个处理器引脚作为 CAN 接收(CAN RX)和 BOOT0 引脚，这就导致了上电时如果 BOOT0 引脚为高电平，处理器将进入引导加载程序模式。在合理的硬件设计中，这种情况应该只在设置引导模式跳线时发生。但在这个设计中，为确保处理器复位时 CAN RX 引脚为低电平，官方使用了晶体管 Q1 和 Q2 来延迟 CAN 收发器的供电，如果没有这些晶体管，处理器将始终以 DFU 模式启动。



在 DFU 模式下，处理器运行其内部的引导加载程序，并准备接收固件更新，此时 CANable 只有红色 LED 指示灯亮起，并且在设备管理器中显示为 DFU 设备，也无法连接到 CAN 总线。这种设计缺陷导致的后果是，短暂中断 5V USB 电源会让 CANable 进入 DFU 模式。也就是说，如果 USB 接口发生了松动或接触不良，亦或是重启计算机，都会导致 CANable 错误地进入 DFU 模式。此外，快速拔插 USB 接口也同样会进入 DFU 模式。这种情况下想要退出 DFU 模式，需要断开 USB 接口 5 秒钟以上，等待 C3 电容释放完电量才能办到。这种设计缺陷让人哭笑不得，因此 CANable 2.5 固件提供了一个解决

方案，即可以选择禁用掉 BOOT0 引脚。

2.3 CANable 官方固件

原版 CANable 适配器使用由 Geschwister Schneider 最初开发的，基于 STM32G431 处理器的开源固件。后来，该固件在 GitHub 上进行了适配，以支持多种处理器，并添加了 CAN FD 支持。目前有两个版本：用于比较旧的处理器的 CANable 1.0 版本和支持 CAN FD 的 CANable 2.0 版本，您可以在 canable.io 上找到网页更新程序，该程序提供两种不同的固件类型。

CANable 2.0 - Updater

- First, hold down the "Boot" button on your CANable 2.0, then plug it in
- Next, select the firmware you want to flash and press "Connect and Update" below
- Nothing showing up in the Chrome connection window on Windows? Run [this utility](#) to fix your drivers.
- Nothing showing up on Linux? You likely need to [add a udev rule](#) or run Chrome as root

slcan with FD support (b158aa7 7/6/22) ▾

Connect and Update

WebUSB not available (you must be running

slcan with FD support (b158aa7 7/6/22)

candlelight (ba6b1dd 7/16/21)

works for hobby but not for professional use

complete garbage, does not work at all

CANable builds are compiled directly from the [candlelight](#) and [slcan](#) repositories on github

关于如何烧录官方原版固件，我们这里不再作描述，请参考我们的往期教程。

2.4 Slcan 协议

当连接 Slcan 固件的 CANable 时，Windows 系统会显示一个 COM 端口。此时与 CAN 总线的所有通信都以 ASCII 字符的形式传输，这是一种通过 USB 接口传输二进制数据的低效方式，发送到 CAN 总线的每个字节都需要通过 USB 传输两个字节。USB 的速度限制为 12 Mbit/s，而 CAN 总线的速度最高可达 10 Mbaud。Slcan 协议的唯一优点是简单易用适合新手。介于其速度较慢，因此不建议用于高速的专业用途。

2.5 Candlelight 协议

Candlelight 协议通过 USB 传输二进制数据，理论上可以充分利用 USB 的速度。但是 canable.io 提供的官方固件仍未实现 CAN FD。此前，市面上还没有完全适用于 STM32G431 处理器的 Candlelight 固件，Candlelight 2.5 是第一个。该固件实现了一种新的协议，可以最大限度地优化 USB 数据传输。此外，Candlelight 2.5 固件仍然 100%向下兼容旧版协议。

2.6 USB 数据传输效率

下表比较了不同固件版本接收 CAN FD 数据包的效率，本例中的有效载荷由 29 位 CAN ID + 16 字节数据 + 标志位 = 21 字节组成。

19F50C02: 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F (FDF, BRS)

固件	Payload	Transferred over USB	效率
官方 Sscan	21 byte	有效载荷 × 2 + 2 = 44 byte	48 %
ElmüSoft Sscan 2.5	21 byte	有效载荷 × 2 + 2 = 44 byte	48 %
官方 Candlelight	21 byte	未实现 CANFD	0 %
ElmüSoft Candlelight 2.5	21 byte	有效载荷 + 2 = 23 byte	91 %

2.7 Candlelight 固件与 Windows 兼容性问题

旧版 Candlelight 固件的 CANable 想要被 Windows 系统正确识别，需要手动安装驱动，但还是会有部分计算机仍然无法正确识别，这个问题一直没有解决方案，我们也只能建议客户改用 Sscan 固件。但是新固件的出现解决了这个问题，在烧录 ElmüSoft Candlelight 2.5 固件时，Windows 会立即安装两个驱动程序，这个过程非常快，甚至难以被察觉。安装过程不会有任何窗口提示，Windows 会在后台悄无声息地完成安装，只需一秒钟。

全新的 Candlelight 2.5 固件是首个在 Windows 系统上 100% 正常运行的 Candlelight 固件。使用 HUD ECU Hacker 打开您的适配器，即可在“Trace”窗格中查看所有详细信息。HUD ECU Hacker 会向 CANable 发送 Microsoft 操作系统描述符请求，并显示适配器的响应。适配器必须正确响应 2 个接口的 3 个请求。

(1) 这里显示的是 CANable 2.5 固件的正确响应：

```
String ID: 0xEE, Signature: "MSFT100", VendorCode: 0x20
Interface: 0, Version: 1.00, MS Compatible ID: "USB\MS_COMP_WINUSB"
Interface: 1, Version: 1.00, MS Compatible ID: "USB\MS_COMP_WINUSB"
Interface: 0, Property: "DeviceInterfaceGUIDs", Type: MultiString, Data: "{c15b4308-04d3-11e6-b3ea-6057189e6443}\0\0"
Interface: 1, Property: "DeviceInterfaceGUIDs", Type: MultiString, Data: "{c25b4308-04d3-11e6-b3ea-6057189e6443}\0\0"
```

接口 0 是 Candlelight 接口，接口 1 是 DFU 固件更新接口。这两个接口都需要一个唯一的标识符(GUID)，以便 Windows 可以为每个接口安装驱动程序。

(2) 这是旧版 Candlelight 1.0 固件的响应，该固件没有版本号：

```
String ID: 0xEE, Signature: "MSFT100", VendorCode: 0x20
Interface: 0, Version: 1.00, MS Compatible ID: "USB\MS_COMP_WINUSB"
```

```
Interface: 1, Version: 1.00, MS Compatible ID: "USB\MS_COMP_WINUSB"
Interface: 0, Property: "DeviceInterfaceGUIDs", Type: MultiString, Data: "{c15b4308-04d3-11e6-b3ea-6057189e6443}\0\0"

Interface: 1, Error requesting MS OS Extended Properties descriptor.
The device has a buggy firmware. The driver for the firmware update (DFU) interface cannot be installed automatically.
```

(3)官方 Candlelight 固件 ba6b1dd 无法响应请求，因此无法安装任何驱动程序。

2.8 全新固件烧录程序

HUD ECU Hacker 内置固件更新程序，适用所有类型的 STM32 处理器。如何烧录固件请参考我们的使用教程。HUD ECU Hacker 总共有四个 CANable 固件可供选择：

File Name	Destination Adapters
STM32G431 - Sican2.5 - MksMakerbase.dfu	MKS Makerbase, DSD Tech, Walfront
STM32G431 - Sican2.5 - OpenlightLabs.dfu	Openlight Labs
STM32G431 - Candlelight2.5 - MksMakerbase.dfu	MKS Makerbase, DSD Tech, Walfront
STM32G431 - Candlelight2.5 - OpenlightLabs.dfu	Openlight Labs

这些适配器之间的唯一区别在于绿色 LED 使用的引脚，Openlight Labs 使用处理器引脚 B11，而其他公司使用引脚 A0。如果安装了错误的固件，绿色 LED 将始终熄灭。

该固件已在 STM32G431 处理器上测试过，但应该也适用于该系列的所有处理器：STM32G441、STM32G471、STM32G473、STM32G474、STM32G483、STM32G484、STM32G491、STM32G4A1。HUD ECU Hacker 固件更新程序可以刷新任何 STM32 处理器，包括比较旧的 CANable 1.0。

刷新固件几乎没有风险，因为该处理器具有硬编码的引导加载程序，不会被轻易擦除。固件烧录完成后，重新上电或退出 DFU 模式，蓝绿色 LED 灯会进行交替闪烁，如果没有，那就表明固件未能正常工作，可能是因为固件上传出现了错误，需要重新烧录。在烧录固件之前，建议使用“Download”按钮，就可以将当前固件保存到 DFU 文件中，以作备份。

固件更新程序使用的是意法半导体(STMicroelectronics)开发的 DFU 文件格式，其中还包含了名称、目标、版本、内存地址等附加信息。如果想要烧录其他固件格式，例如 BIN、HEX 或 S19，可以将其复制到“Firmware”子文件夹中，固件更新程序会自动将这些文件转换为与当前所选设备匹配的 DFU 文件。

在烧录固件之前，还必须选择目标内部闪存。固件更新程序还可以写入选项字节 (Option Bytes)和一次性可编程存储器(OTP)，但不建议新手这么做。

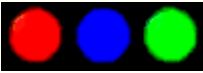
单击“Identify”按钮后，CANable 上的 LED 指示灯会闪烁，再次单击时停止闪烁。如果同时连接了多个适配器，此功能可以帮助区分正在通信的是哪个适配器。

点击“Disable Pin Boot0”会禁用掉 Boot0 引脚，这意味着即使设置了 DFU 模式跳线，

处理器也不会进入 DFU 模式。不过您仍然可以通过单击 “Enter DFU mode”按钮恢复此设置。新的 CANable 2.5 固件里包含了一个进入引导加载程序的命令，也就是说烧录了新固件之后，无需在主板上设置跳线就可以进入 DFU 模式。

2.9 LED 指示灯含义

红色指示灯连接 USB 接口的 5V 电源，用于表示 CANable 的供电状态，不受处理器控制。

LED's	Meaning	LED's	Meaning
	适配器已准备就绪并关闭。		1.) 适配器处于打开状态：无 CAN 总线流量。 2.) 适配器处于固件更新模式。 3.) USB 总线处于睡眠模式下。
	1.) 适配器已打开：RX CAN 总线流量。 2.) 适配器已关闭：已收到 USB 命令。		适配器在发生严重错误后被阻塞：缓冲区溢出或总线关闭。
	适配器已打开：TX CAN 总线流量。		1.) 通电后，两个 LED 灯交替闪烁。 2.) 点击 “识别” 按钮时。
	适配器已打开：RX+TX CAN 总线流量。		

ElmüSoft 修复了 LED 指示灯的行为问题，旧版 Slcan 固件完全不显示 CAN 总线发送 (TX)流量，即使出现错误且数据包未发送，也会使 TX LED 指示灯闪烁。新的 CANable 2.5 固件，两个 LED 指示灯都能准确显示 CAN 总线上发生的情况：

- (1)绿色 TX LED 指示灯仅在 CAN 数据包成功发送并被接收方确认后才会闪烁；
- (2)蓝色 RX LED 指示灯在接收到 CAN 数据包时始终闪烁，即使数据包被过滤器拦截。

2.10 上位机软件

官方建议 CANable 用户使用上位机 Cangaroo，但是它无法计算任何给定波特率/采样点组合的设置。它只为 16 MHz、48 MHz 和 170 MHz 时钟频率的处理器实现了硬编码的表格。Cangaroo 提供的 CAN 总线波特率选项非常有限，采样点选项则更少，当连接其他时钟频率(例如 160 MHz)的适配器时，将无法选择任何波特率。Cangaroo 无法显示任何错误信息。Cangaroo 对 WinUSB 的使用方式不正确，这可能会导致在高总线负载下数据包顺序错误。Cangaroo 甚至还可能突然无限期地卡住甚至崩溃。

现在 CANable 可以在 HUD ECU Hacker 使用了，这是一款高质量的软件。Elmue 表示

这是一款慈善软件，如果您喜欢这款软件，可以向你选择的慈善机构捐款。HUD ECU Hacker 是一款功能强大的工具，拥有众多特性，堪称 CAN 总线和 ECU 的瑞士军刀。

CAN 原始终端允许您查看 CAN 总线流量并手动发送数据包，并支持同时打开多个终端窗口，每个适配器对应一个窗口。您还可以编写宏脚本来接收数据包并发送响应，模拟任何 CAN 控制器。您可以从 CAN 总线流量中获取日志文件，还可以将整个日志文件的内容发送到 CAN 总线。

此外，HUD ECU Hacker 还有一个嗅探终端，它可以在不产生干扰的情况下静默嗅探 CAN 总线流量。在嗅探终端中，数据包通常不会被确认，适配器以静默模式工作。

如果 HUD ECU Hacker 仅仅只能显示通过 CAN 总线传输的原始数据包，那它还称不上全能，HUD ECU Hacker 还可以解码 ISO 15765、J1939 和 NMEA 2000 协议。J1939 协议用于公路车辆、工程车辆、农业车辆和林业车辆，NMEA 2000 协议则用于游艇、帆船、渔船和船舶。提供 DBC 文件给 HUD ECU Hacker，它甚至可以解码任何车辆的 CAN 数据包。(上述的这些上位机功能您都可以在 HUD ECU Hacker 的官方网址了解详情：

<https://netcult.ch/elmue/HUD%20ECU%20Hacker/#J1939>)

2.11 CANbus 设置

关于 CANbus 的设置，我们把它分到了使用教程那个章节，这里不多做说明。

2.12 CAN 总线过滤器

CAN 总线过滤器是旧版本没有的新功能，它可以大幅降低 USB 接口的流量，这有时非常重要，USB 的速度限制为 12 Mbit/s，而 CAN 数据的传输速率最高可达 10 Mbaud，所有您不感兴趣的流量都不会通过 USB 发送。下图显示了一个过滤器，它只允许 CAN ID 范围为 18EE0000 到 18EEFFFF 的数据包通过。

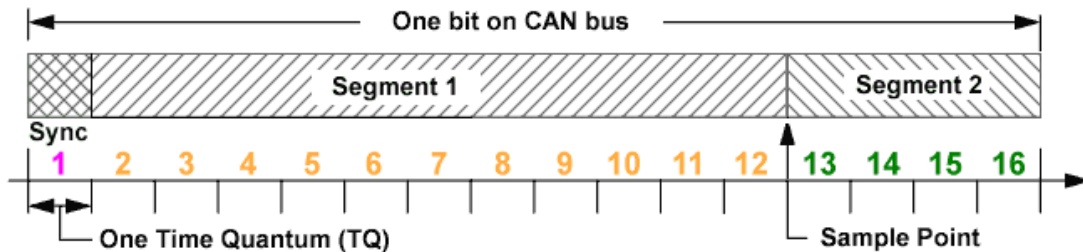
想要了解 CAN 总线过滤器的细节描述，需要您在 HUB ECU Hacker 的手册中查看(手册网址：https://netcult.ch/elmue/HUD%20ECU%20Hacker/#CAN_Filter)

注意：

- ①当接收到 CAN 数据包时，蓝色 LED 会闪烁。当数据包被过滤器拦截时，蓝色 LED 也会闪烁。这意味着，即使您的过滤器拦截了所有数据包，绿色和蓝色 LED 也始终会显示完整的 CAN 总线流量。
- ②STM32 处理器不允许在适配器打开后修改过滤器，唯一的例外是，单个过滤器可以被另一个相同类型的过滤器(11/29 位)替换。

2.13 采样点和波特率

对于传统的 CAN 总线，采样点并不重要。但如果想要使用 CAN FD 总线，就必须输入正确的采样点，否则会出现总线错误。因为在 CAN 总线上，许多控制器必须使用完全相同的设置才能相互同步，正确的采样点对于连接到 CAN FD 总线至关重要。



CAN 数据包中的一位被细分为多个时间片，在本例中为十六个。采样点是处理器测量 CAN 总线状态(高电平或低电平)的精确时刻。采样点必须是两个整数的分数： $1/2 = 50\%$ ， $3/4 = 75\%$ ， $5/8 = 62.5\%$ ， $4/5 = 80\%$ ， $7/8 = 87.5\%$ 。较晚的采样点具有允许使用更长电缆的优势，可以补偿远距离控制器的延迟。较早的采样点的优势在于它对廉价的、不精确的振荡器和抖动的抵抗能力更强。

但主要问题在于 CAN FD 会在 BRS 位采样点精确切换波特率，如果您没有正确输入这两个采样点，则无法与 CAN 总线通信，您可以在 Elmue 的 Oszi 波形分析仪手册中找到更详细的说明以及示波器屏幕截图(<https://netcult.ch/elmue/Oszi-Waveform-Analyzer/#DecodeCAN>)。

处理器需要 3 个设置来配置波特率和采样点：

- ①比特率预分频器：CAN 时钟经预分频器分频后得到一个时间片的时钟信号。
- ②段 1：定义采样点之前的时间片数量（上例中为 11）。
- ③段 2：定义采样点之后的时间片数量（上例中为 4）。

STM32 处理器会单独处理同步段，因此必须将段 1 的值加 1，例如：CAN 时钟 = 160MHz，波特率 = 500kbaud，采样点 = 75%，每比特时间片(TQ) = 16。这可以通过向处理器传递以下设置来实现：预分频器 = 20，段 1 = 11，段 2 = 4。

$$\text{Baudrate} = 160 \text{ MHz} / 20 / (1 + 11 + 4) = 500 \text{ kbaud}$$

$$\text{Samplepoint} = (1 + 11) / (1 + 11 + 4) = 0.75 = 75\%$$

旧版 Sscan 固件对 CAN FD 波特率的选择非常有限：仅支持 2 兆波特和 5 兆波特。旧版 Candlelight 固件理论上可以设置任何可能的波特率。

为了进行计算，主机应用程序必须知道预分频器、Segment1 和 Segment2 设置的范围，而每个处理器对这些值的取值范围都不同，旧版 Candlelight 固件报告的范围是错误的，因此无法进行正确的计算。

摘要：旧版固件均无法根据需要设置精确的波特率/采样点。此外，旧版固件使用的 CAN 时钟频率为 170 MHz，这并非明智之选，应用程序无法从 170 MHz 的频率下导出 2、4、5 或 8 Mbps 的波特率，以及 50%、62.5%、75% 或 87.5% 的采样点。因此新的 CANable 2.5 固件将 CAN 时钟频率更改为 160 MHz。

意法半导体建议标称波特率和数据波特率使用相同的预分频器，波特率就会在 BRS 位采样点处精确切换。而如果波特率未使用相同的预分频器，则会导致相位偏移，从而可能破坏时序。另一个建议是预分频器的值应该尽可能低。

同步跳变宽度应为： $\text{SJW} = \min(\text{Segment1}, \text{Segment2})$ ，SJW 定义了处理器允许接受相位抖动进行重新同步的最大时间间隔。

更多详情请参阅 Documentary 子文件夹中的 PDF 文档 (<https://github.com/Elmue/CANable-2.5-firmware-Sscan-and-Candlelight/tree/main/Documentation>)。

所有这些波特率/采样点相关的设置都非常复杂，而且旧代码中存在很多 bug，所以新固件添加了一个检查机制。主机应用程序会将标称波特率和数据波特率对应的预分频器、

第 1 段和第 2 段的值发送给固件。固件会计算出精确值，并通过调试信息将其返回给主机应用程序。

打开适配器后，您会看到来自固件的以下调试信息：

Nominal: 500k baud, 87.5%; Data: 2M baud, 75.0%; Perfect match: Yes

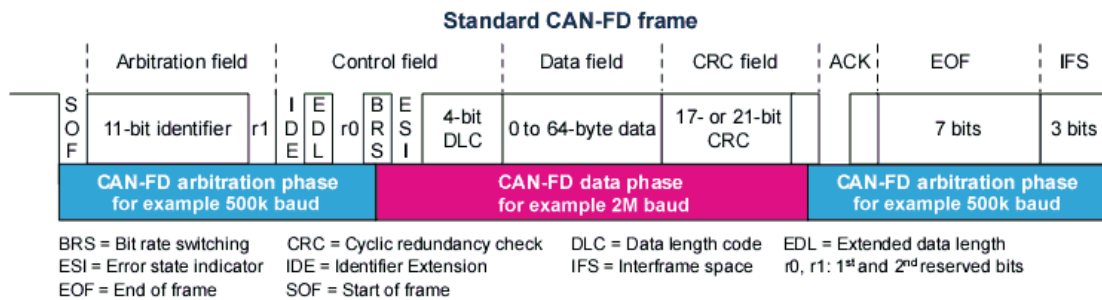
当您看到“完美匹配：是”时，这意味着“标称”和“数据”使用推荐的相同预分频器。注意，要实现上述设置，可以使用多种计算方法，例如：

①标称=[预分频器:2,段 1:139,段 2:20,SJW:20]和数据=[预分频器:2,段 1:29,段 2:10,SJW:10]

②标称=[预分频器:10,段 1:27,段 2:4,SJW:4]和数据=[预分频器:10,段 1:5,段 2:2,SJW:2]

这两个示例都能得到相同的波特率和采样点。但是，第一个示例可以正常工作，而第二个示例在使用 BRS 发送数据包时会产生总线关闭错误！

始终选择尽可能小的预分频器，这样可以获得更精细的重新同步粒度。如果您让 HUD ECU Hacker 为您找到最佳值，则无需关心这些数学计算。HUD ECU Hacker 实现了一套复杂的算法，可以自动完成所有这些计算。您只需输入波特率和采样点，HUD ECU Hacker 就会为您计算出最佳设置。如果某个采样点无法精确匹配，它会显示警告并使用最接近的值。如果某个波特率无法匹配，它会显示错误信息。



2.14 CANbus 错误

旧版 Sscan 固件完全没有错误报告机制，当向 CAN 总线发送数据包时，根本无法确定数据包是否已成功发送。如果在设置波特率时发送无效参数，您将永远无法得知没能连接到 CAN 总线的原因。旧版 Candlelight 固件虽然有错误报告机制，但仅限于 CAN 总线错误，而且存在诸多问题。

新版本固件实现了针对 CAN 总线错误和命令错误的完善错误报告机制，您将永远不会再遇到无法正常工作却又不知原因的情况。CAN 总线上的第一个错误会立即报告。第二个错误会在 100 毫秒后报告，并且仅当第二个错误与之前报告的错误不同时才会报告，这样可以避免像旧版固件那样，向主机发送成千上万个错误信息。如果同一错误持续存在很长时间，则会每隔 3 秒报告一次。

```

21:20:14.788Send Packet
21:20:14.78818EEFF01: 01 02 03 04 05 06 07 08    <— 此数据包已成功发送
21:20:14.79018EEFF01: 01 02 03 04 05 06 07 08    <— 这是发射回波
21:20:18.119Send Packet                            <— 开启自动重传功能
21:20:18.11914A90022: 01 02 03 04 05 06 07 08    <— 该数据包未被确认
21:20:18.120Bus Active, Tx Errors: 8                <— 第一个错误立即发生
21:20:18.220Bus Passive, No ACK received, Tx Errors: 128    <— 100 毫秒后出现第二个错误
21:20:18.738Send Packet
  
```

```
21:20:18.73814A90022: 01 02 03 04 05 06 07 08    <— 该数据包未被确认
21:20:21.216Bus Passive, No ACK received, Tx Errors: 128
21:20:24.212Bus Passive, No ACK received, Tx Errors: 128    <—每隔 3 秒重复出现同样的错误
21:20:27.208Bus Passive, No ACK received, Tx Errors: 128
```

CAN 总线上的所有控制器共享相同的两条数据线，如果只有一个控制器出现故障并违反了 CAN 总线规则，就会扰乱所有其他控制器的通信。因此控制器在检测到问题时关闭其发送器至关重要。这种错误检测必须通过硬件实现，并且必须快速可靠。所有带有 CAN 总线控制器的处理器都会区分以下四种状态：

- ①总线活动（无错误或错误数少于 96 个）
- ②警告级别（错误数在 96 到 128 个之间）
- ③总线被动（错误数在 128 到 248 个之间）
- ④总线关闭（错误数超过 248 个）

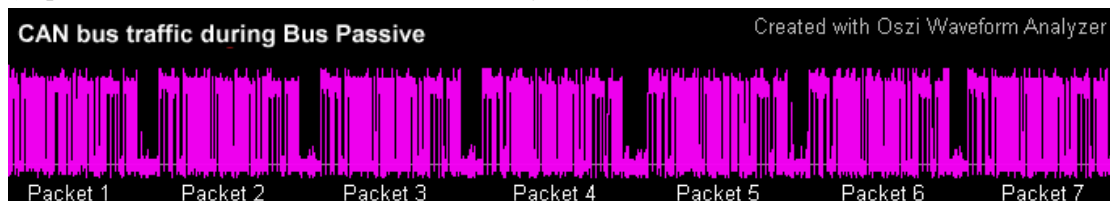
在仲裁阶段，允许多个控制器同时发送数据。其中一个控制器将赢得仲裁。但在数据阶段，只能有一个控制器发送数据。如果处理器检测到严重的数据损坏，它会立即停止发送并进入总线关闭状态。总线关闭是一个严重的错误，可以通过在配置了不同波特率的两个适配器之间发送数据包来创建。

```
22:21:24.187Send Packet
22:21:24.187170AA3CB: 01 02 03 04 05 06 07 08
22:21:24.623Bus Passive, Bit stuffing error, Tx Errors: 128, Rx Errors: 8    <—发送发生损坏
22:21:27.618Bus Passive, Recessive bit error, Tx Errors: 128, Rx Errors: 8
22:21:30.367Bus Off, Frame format error, Tx Errors: 248, Rx Errors: 127    <—接收发生损坏
```

只要稍有经验，你就可以通过了解这些错误的含义来推断 CAN 总线上的问题所在。如果出现严重错误，例如总线关闭或缓冲区溢出，绿色和蓝色 LED 指示灯将常亮。

2.15 发送超时

如果启用了自动重传功能，且处理器未收到确认(ACK)，则会进入发送总线被动状态。在此状态下，处理器会无限重传发送 FIFO 中的第一个数据包，导致总线负载高达 95%。“总线被动”这个名称完全具有误导性。总线并非处于被动状态，而是被大量数据淹没。在示波器上，您可以看到同一个数据包被反复重传，几乎没有停顿 (<https://netcult.ch/elmue/Oszi-Waveform-Analyzer/>)。



因此，Elmue 实现了一个交易超时机制。待处理的交易请求会在 500 毫秒后被取消。您会看到一条错误报告：

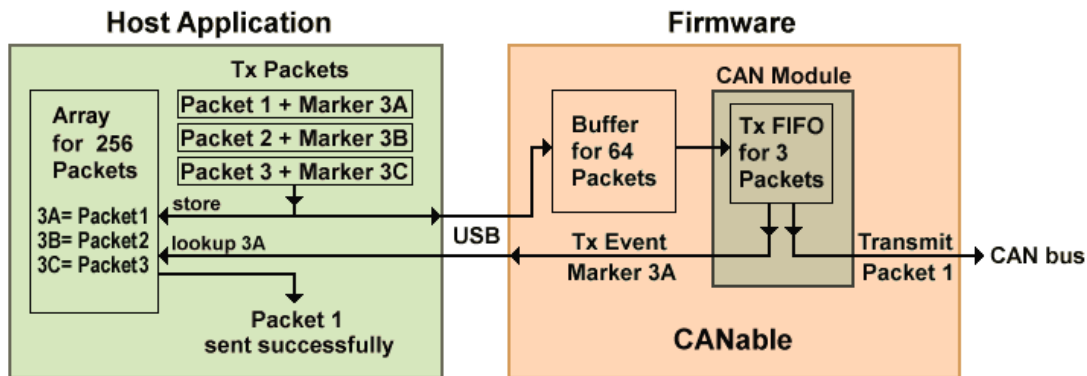
```
Bus Passive, Tx Timeout, No ACK received, Tx Errors: 128
```

错误计数器在达到 128 时停止计数，但如果没有超时，可能会出现数千次传输失败的尝试。

2.16 发射回波标记

旧版 Sscan 固件从未提供过数据包是否已发送的反馈。

旧版 Candlelight 固件的 Tx 数据包反馈机制设计存在缺陷：当固件接收到 Tx 数据包时，它会将整个 Tx 数据包以虚假事件的形式发送回主机，无论数据包是否真的发送到了 CAN 总线，这种反馈都是即时的。对于一个包含 8 个数据字节的 CAN FD 数据包，固件会通过 USB 向主机发送一个 80 字节的虚假事件。这些虚假事件会产生大量无用的 USB 流量，无法关闭，而且时间戳也是错误的。



新固件可以从主机应用程序接收每个发送数据包的 8 位标记。当数据包成功发送到 CAN 总线后，处理器会触发一个包含该标记的 Tx 事件。主机应用程序必须将最后发送的数据包存储在内部数组中(请参考演示应用程序的代码，见 4.1 章节)。适配器最多可存储 $64 + 3 = 67$ 个数据包。8 位标记的值范围为 0 到 255，足以为每个 Tx 数据包提供一个唯一的标记，其优势在于用户可以可靠地反馈哪些数据包已实际发送。在单次传输模式(无重传)下，这是判断数据包是否已被确认的唯一方法。用户还可以查看数据包发送到 CAN 总线的确切时间。通过使用标记，只需将一个字节而非整个数据包传输回主机(参见 3.4 章节的示例)。

2.17 时间戳

一些 Sscan 固件会通过 USB 向主机应用程序发送时间戳，每个接收到的数据包都会附带一个 8 个 ASCII 字符组成的 32 位的时间戳。使用 ASCII 协议本身效率就不高(见 2.6 章节)，但通过在每个数据包中添加额外的 8 个字节来进一步降低 USB 传输速度并不是一个好办法。

CANable 2.5 固件已经尽可能地优化了速度，因此 Sscan 特意没有实现时间戳功能，但为了向后兼容，保留了 1 微秒的时间戳，这些时间戳仅占用 4 个字节，并且可以关闭。

不过更建议使用计算机 CPU 中性能计数器生成的时间戳(<https://learn.microsoft.com/en-us/windows/win32/api/profileapi/nf-profileapi-queryperformancecounter>)，64 位计数器的精度是纳秒级的。当数据包通过 USB 接口送达时，您可以为其添加时间戳。C++ 演示应用程序(见 4.1 章节)展示了如何使用固件和性能计数器生成的时间戳。

2.18 DFU 模式

新的 Sscan 和 Candlelight 固件新增了一个进入 DFU 模式的命令，不再需要启动模式跳线，固件在进入启动模式前会启用 BOOT0 引脚（如果之前被禁用的话）。如果 BOOT0 引

脚被禁用，处理器在收到硬件复位指令之前不会进入启动模式。这意味着只有在 BOOT0 引脚之前被禁用的情况下，用户才必须重新连接 USB 接口。

与旧版固件不同，新版固件在进入启动模式前会有 300 毫秒的延迟。这确保始终会向主机发送反馈，告知命令是否成功。

2.19 Bug 修复

新固件修复了旧固件中的几个漏洞，其中最严重的漏洞包括：

- ①Candlelight 固件在缓冲区溢出后可能会完全崩溃；
- ②Candlelight 的设计存在缺陷：它只使用一个缓冲区来处理 CAN 接收和发送，这导致在发生 CAN 缓冲区溢出后，它甚至无法向主机发送错误状态；
- ③Candlelight 无法正确发送 USB 数据，大小正好为 64 字节的 USB 数据包送达时格式会发生错误。

2.20 总线负载

新固件可以计算总线负载，并以百分比形式显示，显示间隔可由用户定义。HUD ECU Hacker 会在“跟踪”窗格中每 5 秒显示一次总线负载(如果负载不为零)。总线负载的计算相当复杂，可能存在 $\pm 10\%$ 的偏差。

2.21 收发器延迟

CAN 总线收发器芯片的延迟对于高于 1 兆波特的波特率非常重要，处理器会自动测量延迟，固件会将其报告。在发送第一个设置了 BRS 标志的 CAN FD 数据包后，会看到一条调试信息：

```
23:32:21.06919858A01: 0C 00 00 00 F0 18 51 0E (FDF, BRS)
23:32:21.073Measured transceiver chip delay: 131 ns
23:32:21.07319858A01: 0C 00 00 00 F0 18 51 0E (FDF, BRS)
```

通常，收发器芯片的延迟约为 50 纳秒，但在这里看到的 MKS Makerbase 中的独立芯片延迟更低。

第 3 部分 Sscan 应用程序开发手册

本手册适用于希望通过计算机与 CANable 2.5 通信的软件开发人员。Sscan 使用 USB CDC 接口，所有命令和响应均以 ASCII 字符串形式发送，速度较慢(参考 2.6 章节)。因此对于专业应用，建议使用 Candlelight 而非 Sscan。

第一个字符是要执行的指令，最后一个字符始终是回车符。

3.1 Sscan 指令

指令	状态	版本	含义	补充说明
"A1\r"	关闭	原版	启用自动重传	重传数据包直至收到确认
"A0\r"	关闭	原版	禁用自动重传	启用单次发送模式
"C\r"	开启/关闭	原版	关闭适配器	关闭 CAN 连接，重置为默认设置
"L7\r"	开启/关闭	100	启用每 700 毫秒一次的总线负载报告	L1: 报告间隔= 100 毫秒， L50: 间隔= 5 秒 有效间隔范围：0 至 100（最长 10 秒）
"L0\r"	开启/关闭	100	禁用总线负载报告	
"O\r"	关闭	原版	开启适配器	以 M0 / M1 设置的模式连接到 CAN 总线
"ON\r"	关闭	100	以正常模式打开	忽略带有 M0 / M1 的设置
"OS\r"	关闭	100	以静默模式打开	忽略带有 M0 / M1 的设置
"OI\r"	关闭	100	以内部环回模式打开	忽略带有 M0 / M1 的设置
"OE\r"	关闭	100	以外部环回模式打开	忽略带有 M0 / M1 的设置
"V\r"	开启/关闭	原版	返回详细信息：版本/开发板/MCU/限制	返回七个键值对，请参阅版本信息(3.3 章节)。
设置模式	状态	版本	含义	补充说明

"M1\r"	关闭	原版	启用静默模式	下一个命令 "O\r"将以总线监视模式打开。
"M0\r"	关闭	原版	禁用静默模式	下一个命令 "O\r"将以正常模式打开
"MA\r"	关闭	100	启用自动重传模式(与 A1 相同)	重传数据包直至收到确认
"Ma\r"	关闭	100	禁用自动重传模式(与 A0 相同)	启用单次发送模式
"MD\r"	开启/关闭	100	启用调试消息	固件向主机发送字符串消息
"Md\r"	开启/关闭	100	禁用调试消息	不发送调试消息
"ME\r"	开启/关闭	100	启用 CAN 错误报告	CAN 错误信息会发送给主机
"Me\r"	开启/关闭	100	禁用 CAN 错误报告	弃用 CAN 错误报告功能
"MF\r"	开启/关闭	100	启用反馈模式	每个命令都发送成功/错误反馈
"Mf\r"	开启/关闭	100	关闭反馈模式	弃用命令反馈模式
"MI\r"	开启/关闭	100	启用识别模式	绿/蓝 LED 灯开始闪烁
"Mi\r"	开启/关闭	100	禁用识别模式	LED 灯停止闪烁
"MM\r"	开启/关闭	100	启用 Tx 回声报告标记	报告所有成功发送的数据包，并添加标记
"Mm\r"	开启/关闭	100	禁用 Tx 回声报告标记	无 Tx Echo 报告
"MR\r"	开启/关闭	100	启用 120Ω 终端电阻	仅少数主板支持
"Mr\r"	开启/关闭	100	禁用 120Ω 终端电阻	
"MS\r"	开启/关闭	100	启用 ESI 报告	报告接收到的 CAN FD 消息的 ESI 标志
"Ms\r"	开启/关闭	100	禁用 ESI 报告	无 ESI 报告

"MDEFMS\r"	开启/关闭	100	启用调试、错误、反馈、回显、ESI 报告	您可以使用一条命令一次性设置所有模式
设置波特率	状态	版本	含义	补充说明
"S0\r"	关闭	原版	设置标称波特率为 10kbaud	采样点 87.5%
"S1\r"	关闭	原版	设置标称波特率为 20kbaud	采样点 87.5%
"S2\r"	关闭	原版	设置标称波特率为 50kbaud	采样点 87.5%
"S3\r"	关闭	原版	设置标称波特率为 100kbaud	采样点 87.5%
"S4\r"	关闭	原版	设置标称波特率为 125kbaud	采样点 87.5%
"S5\r"	关闭	原版	设置标称波特率为 250kbaud	采样点 87.5%
"S6\r"	关闭	原版	设置标称波特率为 500kbaud	采样点 87.5%
"S7\r"	关闭	原版	设置标称波特率为 800kbaud	采样点 87.5%
"S8\r"	关闭	原版	设置标称波特率为 10Mbaud	采样点 87.5%
"S9\r"	关闭	原版	设置标称波特率为 83.333kbaud	采样点 87.5%
"Y0\r"	关闭	100	设置 CAN FD 数据波特率为 500kbaud	采样点 75% 已弃用
"Y1\r"	关闭	100	设置 CAN FD 数据波特率为 1Mbaud	采样点 75% 已弃用

"Y2\r"	关闭	原版	设置 CAN FD 数据波特率为 2Mbaud	采样点 75% 已弃用
"Y4\r"	关闭	100	设置 CAN FD 数据波特率为 4Mbaud	采样点 75% 已弃用
"Y5\r"	关闭	原版	设置 CAN FD 数据波特率为 5Mbaud	采样点 75% 已弃用
"Y8\r"	关闭	100	设置 CAN FD 数据波特率为 8Mbaud	采样点 50% (75%不工作) 已弃用
"s4,69,10,7\r"	关闭	100	设置标称比特率: 预分频器 =4, Seg1=69, Seg2=10, 同步跳转宽度=7	设置 500 kbaud, 采样点 87.5% 见 2.13 章节
"y4,9,10,7\r"	关闭	100	设置数据比特率: 预分频器 =4, Seg1=9, Seg2=10, 同步跳转宽度=7	设置 2 Mbaud, 采样点 50% 见 2.13 章节
设置过滤器	状态	版本	含义	补充说明
"F7E8,7FF\r"	关闭	100	设置一个仅用于单个 ID 的掩码过滤器: 7E8(11 位)	您最多可以设置 8 个掩码过滤器, 以分号分隔 11 位和 29 位过滤器可以混合使用 请参阅 2.12 章节
"F18DA00F1,1FFF00FF\r"	关闭	100	为 256 个 ID 设置掩码过滤器: 18DAXXF1(29 位)	
"F7E0,7F0;720,7F0;730,7F0\r"	关闭	100	设置 3 个过滤器, 每个过滤器可处理 16 个 ID: 7EX、72X 和 73X。	
"f\r"	关闭	100	清除所有筛选条件	移除所有筛选条件
Boot 模式	状态	版本	含义	补充说明
"*Boot0:Off\r"	关闭	100	禁用引脚 BOOT0	见 2.2 章节

"*Boot0:?\r"	开启/关闭	100	请求引脚 BOOT0 的状态	如果启用，则返回 "+1\r"；如果禁用，则返回 "+0\r"。
"*DFU\r"	开启/关闭	100	启用引脚 BOOT0 并进入 DFU 模式	如果反馈 = FBK_ResetRequired: 重新连接 USB 电缆
发送数据包	状态	版本	含义	补充说明
"Txxxxxxxx\r"	开启	原版	发送带有 29 位 ID 的经典数据包	Bits: IDE 见 3.4 章节
"txxxxxxxxx\r"	开启	原版	发送带有 11 位 ID 的经典数据包	Bits: None
"Rxxxxxxxx\r"	开启	原版	发送带有 29 位 ID 的远程传输请求	Bits: RTR + IDE
"rxxxxxxxx\r"	开启	原版	发送带有 11 位 ID 的远程传输请求	Bits: RTR
"Dxxxxxxxx\r"	开启	原版	发送 CAN FD 数据包, 29 位, 无波特率切换	Bits: FDF + IDE
"dxxxxxxxx\r"	开启	原版	发送 CAN FD 数据包, 11 位, 无波特率切换	Bits: FDF
"Bxxxxxxxx\r"	开启	原版	发送 CAN FD 数据包, 29 位, 带波特率开关	Bits: FDF + BRS + IDE
"bxxxxxxxx\r"	开启	原版	发送 CAN FD 数据包, 11 位, 带波特率开关	Bits: FDF + BRS

注意：请勿将 S 和 Y 命令用于 CAN FD 总线，它们无法选择正确的采样点。您可以使用它们在桌面上的两个适配器之间进行测试，但如果要连接到实际的 CAN FD 总线，则不能使用它们。例如，如果您要连接到使用 CAN FD 总线的汽车，则必须知道 CAN 总线使用的两个采样点。然后必须根据所需的波特率和采样点计算预分频器、段 1 和段 2 的值，并使用 S 和 Y 命令。如果使用错误的设置连接到 CAN FD 总线，您将收到“总线被动”甚至“总线关闭”错误(参阅 2.13 章节)。

3.2 Sican 响应和事件

反馈	版本	含义	补充说明
"#\r"	100	命令已成功执行	需要启用反馈模式
"#1\r"	100	命令无效	需要启用反馈模式
"#2\r"	100	命令的某个参数无效	需要启用反馈模式
"#3\r"	100	该命令要求适配器处于开启状态	需要启用反馈模式
"#4\r"	100	该命令要求适配器处于关闭状态	需要启用反馈模式
"#5\r"	100	意法半导体的 HAL 报告了一个错误	需要启用反馈模式
"#6\r"	100	该功能主板不支持/尚未实现	需要启用反馈模式
"#7\r"	100	CAN 发送缓冲区已满(未收到 ACK, 有 67 个数据包等待)	需要启用反馈模式
"#8\r"	100	CAN 总线已关闭(发生严重的 CAN 错误)	需要启用反馈模式
"#9\r"	100	在静默模式下无法发送数据包	需要启用反馈模式
"#:\r"	100	尚未设置所需的波特率	需要启用反馈模式
"#;\r"	100	对闪存中的选项字节进行编程失败	需要启用反馈模式
"#<\r"	100	请重新连接 USB 连接线, 需要进行硬件重置	需要启用反馈模式
响应	版本	含义	补充说明
" +Text\r"	100	固件会向命令发送文本响应	用于命令 "*Boot0:?\r" 和 "\V\r"
事件	版本	含义	补充说明
">Message\r"	100	固件发送调试信息(纯文本)	需要启用调试消息
"Exxxxxxxx\r"	100	固件会报告 CAN 错误状态(见 3.5 章节)	需要启用 CAN 错误报告

"L27\r"	100	固件计算出的总线负载为 27%， 如果总线负载为零，则不发送报告	需要启用 canbus 负载报告。
"M3C\r"	100	固件报告 Tx 回显标记 0x3C(见 3.4 章节)	需要启用 Tx Echo Report 标记
Rx Packets	Version	Meaning	Comment
"Txxxxxxxx\r"	原版	收到带有 29 位 ID 的经典数据包	Bits: IDE (见 3.4 章节)
"txxxxxxxxx\r"	原版	收到带有 11 位 ID 的经典数据包	Bits: None
"Rxxxxxxxx\r"	原版	收到带有 29 位 ID 的远程传输请求	Bits: RTR + IDE
"rxxxxxxxx\r"	原版	收到带有 11 位 ID 的远程传输请求	Bits: RTR
"Dxxxxxxxx\r"	原版	已收到 CAN FD 数据包, 29 位, 无波特率切换	Bits: FDF + IDE
"dxxxxxxxx\r"	原版	已收到 CAN FD 数据包, 11 位, 无波特率切换	Bits: FDF
"Bxxxxxxxx\r"	原版	已收到 CAN FD 数据包, 29 位, 带波特率开关	Bits: FDF + BRS + IDE
"bxxxxxxxx\r"	原版	已收到 CAN FD 数据包, 11 位, 带波特率开关	Bits: FDF + BRS

3.3 Slcan 版本信息

在新固件中，命令 “\r” 返回一个包含七个键值对的字符串，键值对之间以制表符分隔。它以加号 (+) 开头，表示命令响应包含文本内容，而不是反馈响应 (#)。

```
+Board: MksMakerbase\t
MCU: STM32G431\t
DevID: 1128\t
Firmware: 2427156\t
Slcan: 100\t
Clock: 160\t
Limits: 512,256,128,128,32,32,16,16\r
```

将此字符串在每个制表符(\t)处分割，然后再在每个部分以冒号处分割，将得到 7 个键值对：

关键字	值	补充说明
Board	MksMakerbase	这定义了固件是为哪个开发板编译的，如果用户上传了错误的固件，则可能与实际硬件不匹配，见 2.8 固件版本表格
MCU	STM32G431	这定义了固件编译时所针对的处理器，如果用户上传了错误的固件，则可能与实际处理器不匹配，见 2.8 固件版本表格
DevID	1128	这是实际处理器的“Device ID”。将 1128 转换为十六进制，得到 0x468，每个处理器系列都由一个唯一的“Device ID”标识，在意法半导体(STMicroelectronics)的参考手册中，您可以找到以下标识符： 0x468 = 2 类设备 (STM32G431、STM32G441) 0x469 = 3 类设备 (STM32G471、STM32G473、STM32G474、STM32G483、STM32G484) 0x479 = 4 类设备 (STM32G491、STM32G4A1)
Firmware	2427156	旧版固件返回的版本号完全没有意义，例如“ba6b1dd”。这里显示的版本号表示固件的创建日期，将其转换为十六进制后，得到 BCD 版本号 0x250914，表示 2025 年 9 月 14 日
Sican	100	这是您在上方表格“版本”列中看到的 Sican 模块版本。ElmüSoft 创建的第一个版本是版本 100，其中包含此处描述的所有功能。未来的版本将是 101、102 等，并包含新的命令，这些命令将在此处记录。使用此版本号来确定固件支持哪些功能。
Clock	160	CAN 时钟频率为 160MHz
Limits	512,256,128,128,32,32,16,16	这 8 个值是比特率命令“s”和“y”的上限。每个处理器对预分频器、Segment1、Segment2 和同步跳转宽度(SJW)的限制各不相同。 标称波特率：最大预分频器 = 512，最大 Segment1 = 256，最大 Segment2 = 128，最大 SJW = 128 FD 数据波特率：最大预分频器 = 32，最大 Segment1 = 32，最大 Segment2 = 16，最大 SJW = 16 见 2.13 章节

3.4 Sllcan 数据包

所有数据包均按以下模式传输：

- ①数据包类型：T、t、R、r、D、d、B 或 b（参见上表）；
- ②CAN ID：11 位 CAN ID 为 3 位数字，29 位 CAN ID 为 8 位数字；
- ③DLC：1 位数字（数据字节数）
经典数据包：DLC = '0'至'8'；
CAN FD：'8' = 8 字节，'9' = 12 字节，'A' = 16 字节，'B' = 20 字节，'C' = 24 字节，'D' = 32 字节，'E' = 48 字节，'F' = 64 字节；
- ④数据字节数：经典数据包：0 至 8，CAN FD：8 至 64 字节；
- ⑤仅适用于发送数据包，且仅当启用发送回显标记时：一个 2 位数字标记；
- ⑥仅适用于接收数据包，且仅当启用 ESI 报告时：如果发送节点是错误被动的，则显示字符“S”。

方向	数据包	Sllcan	补充说明
双向	18AABBCC: 11 22 33 44 55	"T18AABBCC51122334455\r"	CAN, 29 bit ID
双向	7E0: 11 22 33 44 55 66 77 88 99 AA BB CC	"d7E09112233445566778899AABBCC\r"	CAN FD, 11 bit, FDF
接收	7E0: 11 22 33 44 55 66 77 88	"b7E081122334455667788S\r"	FDF, BRS, ESI
发送	7E0: 11 22 33 44 55 66	"t7E061122334455663F\r"	Marker = 3F

注意：如果数据包后面有任何额外的字符，则表示语法错误，示例：

```

Tx: "t7E061122334455663F\r" <—发送带有标记 3F 的数据包
Rx: "#\r" <—收到命令反馈（成功）
Rx: "M3F\r" <—接收回声标记 3F

```

3.5 Sllcan 错误报告

启用 CAN 总线错误报告后，至少出现一个错误时才会收到错误报告。错误报告会立即发送、在 100 毫秒或 3 秒后发送，错误报告的格式始终为“Exxxxxxx\r”，包含 5 个部分：

Digit	Meaning	Value
1	CANbus 状态	'0' = Bus Active '1' = Warning Level '2' = Bus Passive '3' = Bus Off
2	最新协议错误	'0' = None '1' = Bit 填充错误 '2' = 帧格式错误 '3' = 未收到确认信息 '4' = 隐性比特错误 '5' = 主导比特错误 '6' = CRC 错误
3 + 4	固件错误标志	0x00 = None 0x01 = Rx 失败 0x02 = Tx 失败 0x04 = CAN 发送缓冲区溢出 0x08 = USB 输入缓冲区溢出 0x10 = Tx 超时 可以组合使用多个标志
5 + 6	Tx 错误数	0 ... 255 errors
7 + 8	Rx 错误数	0 ... 255 errors

例如：“E2310800C\u”表示：总线处于被动状态，未收到 ACK，发送超时，发送错误 128 次，接收错误 12 次。

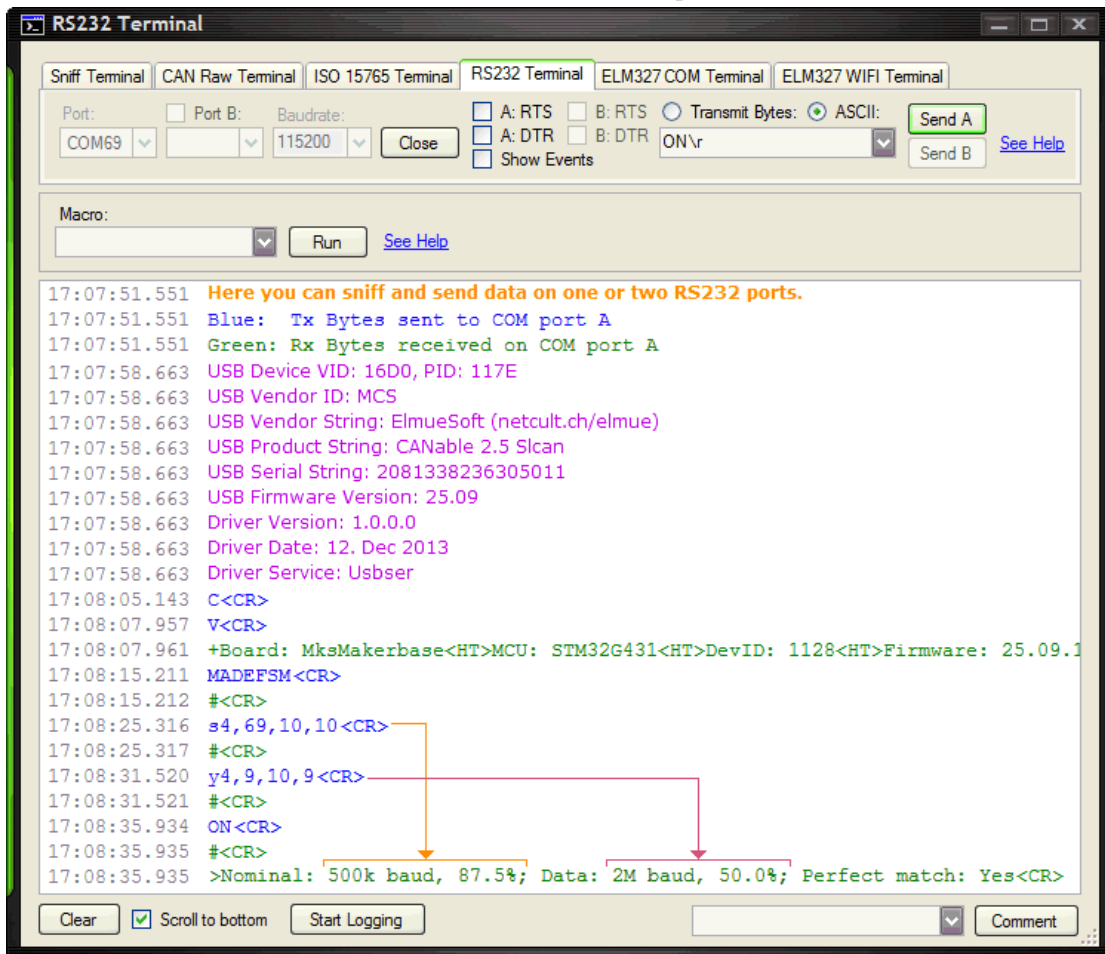
3.6 Sican 初始化

连接到 CANable 2.5 适配器时，应按以下步骤操作：

- 1、如果设备仍处于打开状态，请发送命令 “C\u”以关闭设备。但此命令还有另一个用途：它会将固件中的所有内部变量重置为默认值，波特率和所有模式都会重置(恢复到传统

模式), 报告功能将被关闭, 滤波器将被移除, 接收/发送缓冲区也会被清空。注意: 尽管反馈功能已启用, “Close”命令是唯一不发送反馈的命令。这样做的目的是让您始终可以将其作为第一个命令发送, 而无需等待反馈。连接到 CANable 时, 您无法确定用户连接的是传统设备还是 CANable 2.5。此外, 固件此时也无法确定您是否稍后会启用反馈模式。因此, “Close”命令的行为与传统命令完全相同: 它永远不会发送反馈。

- 2、发送命令 “V\r”以获取版本字符串, 在制表符处分割响应字符串。如果收到 7 个或更多部分, 则表示您正在与 CANable 2.5 通信, 否则使用的是旧版固件。如果您不支持旧版设备, 则显示错误消息, 需要切换到旧版模式。如果是 CANable 2.5, 请使用 Sscan 版本号(100 或更高)来检查哪些功能可用。
- 3、发送命令 “MF\r”以启用反馈模式(或发送“MADEFMS\r”以启用您需要的所有其他模式)。这将是第一个发送反馈的命令, 在此之后, 您必须等待反馈 “#\r”并检查错误, 然后再发送下一个命令。
- 4、发送命令 “S”或“s”以设置标称波特率。
- 5、如果您还发送命令 “Y”或“y”来设置数据波特率, 则固件将自动切换到 CAN FD 模式。
- 6、然后您可以选择设置过滤器。
- 7、最后执行命令 “ON\r”以正常模式打开设备。下图显示的是手动发送到适配器的初始化序列, 需使用 HUD ECU Hacker 中的 RS232 终端。您输入的用于打开 COM 端口的波特率无关紧要。该端口始终以最大 USB 速度(12 Mbps)运行。



```
RS232 Terminal
Sniff Terminal | CAN Raw Terminal | ISO 15765 Terminal | RS232 Terminal | ELM327 COM Terminal | ELM327 WIFI Terminal
Port: COM69 | Baudrate: 115200 | A: RTS | B: RTS | Transmit Bytes: ASCII | Send A
A: DTR | B: DTR | ON\r | Send B | See Help
Show Events
Macro: | Run | See Help
17:07:51.551 Here you can sniff and send data on one or two RS232 ports.
17:07:51.551 Blue: Tx Bytes sent to COM port A
17:07:51.551 Green: Rx Bytes received on COM port A
17:07:58.663 USB Device VID: 16D0, PID: 117E
17:07:58.663 USB Vendor ID: MCS
17:07:58.663 USB Vendor String: ElmueSoft (netcult.ch/elmue)
17:07:58.663 USB Product String: CANable 2.5 Sscan
17:07:58.663 USB Serial String: 2081338236305011
17:07:58.663 USB Firmware Version: 25.09
17:07:58.663 Driver Version: 1.0.0.0
17:07:58.663 Driver Date: 12. Dec 2013
17:07:58.663 Driver Service: Usbser
17:08:05.143 C<CR>
17:08:07.957 V<CR>
17:08:07.961 +Board: MksMakerbase<HT>MCU: STM32G431<HT>DevID: 1128<HT>Firmware: 25.09.1
17:08:15.211 MADEFMS<CR>
17:08:15.212 #<CR>
17:08:25.316 s4,69,10,10<CR>
17:08:25.317 #<CR>
17:08:31.520 y4,9,10,9<CR>
17:08:31.521 #<CR>
17:08:35.934 ON<CR>
17:08:35.935 #<CR>
17:08:35.935 >Nominal: 500k baud, 87.5%; Data: 2M baud, 50.0%; Perfect match: Yes<CR>
Clear | Scroll to bottom | Start Logging | Comment
```

第 4 部分 Candlelight 应用程序开发手册

Candlelight 2.5 固件可用于构建专业的高速 CAN FD 应用，Candlelight 固件的控制比 Sllcan 固件更为复杂。其中包含大量的结构体和枚举，如果在此一一解释，本手册篇幅将非常庞大。

4.1 C++ 演示应用程序

Elmue 编写了一个可直接使用的 C++ Candlelight 类，您可以将其复制粘贴到您的项目中，这个功能强大且经过充分测试的类演示了新固件提供的所有功能：

检测并枚举所有已连接的 Candlelight 设备；

获取已连接设备的详细信息：USB 描述符、板卡、处理器、版本号等；

设置波特率和采样点；

设置掩码过滤器；

打开/关闭/启动/重置；

发送数据包/接收数据包/处理 Tx 回显标记；

解码并显示 CAN 总线错误；

禁用启动引脚/进入 DFU 模式；

源代码包含详细的注释，方便您轻松理解其工作原理，该 C++ 示例还展示了如何正确使用 WinUSB.dll，这比乍看之下要复杂得多。例如，Cangaroo 和 Candle.NET 对 WinUSB 的使用方式不正确，导致数据包丢失或数据包顺序错误，WinUSB 需要一种微软文档中未提及的特殊编程方式。

```
lime = Sent packets
Green = Echo of sent packets that have been ACKnowledged
Cyan = Received packets

19:26:49.941.342 Info Pin B00T0 is disabled
19:26:49.965.330 Debg Nominal: 500k baud, 87.5%; Data: 2M baud, 75.0%; Perfect match: Yes
19:26:51.925.359 Send 7E0: 45 6C 6D 75 53 6F 66 74
19:26:51.926.354 Echo 7E0: 45 6C 6D 75 53 6F 66 74
19:26:54.000.541 Send 7E0: 46 8C 18 B0 F0 90 F0 D0
19:26:54.001.317 Echo 7E0: 46 8C 18 B0 F0 90 F0 D0
19:26:55.077.290 Recv 701: 9A 4E 77 02 68 9E 75 22 19 FF E4 00 FDF BRS
19:26:56.058.521 Send 7E0: 47 8E 1C 78 18 A8 18 C8
19:26:56.059.186 Echo 7E0: 47 8E 1C 78 18 A8 18 C8
19:26:58.133.629 Send 7E0: 48 90 20 40 40 C0 40 C0
19:26:58.135.095 Echo 7E0: 48 90 20 40 40 C0 40 C0
19:27:00.208.028 Send 7E0: 49 92 24 08 68 D8 68 B8
19:27:00.209.116 Echo 7E0: 49 92 24 08 68 D8 68 B8
19:27:02.282.755 Send 7E0: 4A 94 28 D0 90 F0 90 B0
19:27:02.284.015 Echo 7E0: 4A 94 28 D0 90 F0 90 B0
19:27:02.892.068 Recv 701: 0C 00 00 00 F0 18 51 0E
19:27:04.310.717 Send 7E0: 4B 96 2C 98 B8 08 B8 A8
19:27:04.311.970 Echo 7E0: 4B 96 2C 98 B8 08 B8 A8
19:27:06.385.452 Send 7E0: 4C 98 30 60 E0 20 E0 A0
19:27:06.386.844 Err Bus Active, Tx Errors: 8
19:27:06.485.849 Err Bus Passive, No ACK received, Tx Errors: 128
19:27:06.884.869 Err Bus Passive, Tx Timeout, No ACK received, Tx Errors: 128
19:27:06.984.898 Err Bus Passive, Tx Errors: 128
19:27:08.398.257 Send 7E0: 4D 9A 34 28 08 38 08 98
19:27:08.399.779 Err Bus Passive, No ACK received, Tx Errors: 128
19:27:08.897.766 Err Bus Passive, Tx Timeout, No ACK received, Tx Errors: 128
19:27:08.997.858 Err Bus Passive, Tx Errors: 128
19:27:10.410.141 Send 7E0: 4E 9C 38 F0 30 50 30 90
19:27:10.411.731 Err Bus Passive, No ACK received, Tx Errors: 128
19:27:10.909.705 Err Bus Passive, Tx Timeout, No ACK received, Tx Errors: 128
19:27:11.009.782 Err Bus Passive, No ACK received, Tx Errors: 128
19:27:11.109.752 Err Bus Passive, Tx Errors: 128
19:27:12.422.512 Send 7E0: 4F 9E 3C B8 58 68 58 88
19:27:12.423.701 Err Bus Warning, Tx Errors: 127
19:27:12.424.645 Echo 7E0: 4F 9E 3C B8 58 68 58 88
19:27:14.513.490 Send 7E0: 50 A0 40 80 80 80 80 80
19:27:14.514.683 Echo 7E0: 50 A0 40 80 80 80 80 80
19:27:14.515.563 Err Bus Warning, Tx Errors: 126
19:27:16.603.812 Send 7E0: 51 A2 44 48 A8 98 A8 78
19:27:16.604.726 Err Bus Warning, Tx Errors: 125
19:27:16.605.514 Echo 7E0: 51 A2 44 48 A8 98 A8 78
19:27:18.694.198 Send 7E0: 52 A4 40 10 D0 B0 D0 70
19:27:18.695.572 Err Bus Warning, Tx Errors: 124
```

CAN FD packet received

Here the listening adapter has been turned off.

Here the listening adapter has been turned back on.

4.2 新的 ElmüSoft 协议

传统的 Candlelight 协议存在一些设计缺陷，例如，当向主机发送 CAN FD 数据包时，时间戳被放置在一个固定长度为 64 字节的数据数组之后。这种不必要的举动导致即使数据包只有 8 个字节，也始终需要通过 USB 发送 80 个字节的数据。此外，传统的 Candlelight 协议最初设计的目的就是传输 CAN 数据包以外的任何数据，但它无法向主机传输其他数据，例如 ASCII 调试字符串。

因此，我不得不设计一种新的高效协议：ElmüSoft 协议。它在通过 USB 传输时，仅在 CAN 数据包有效载荷中添加 2 个额外的字节。它采用极简的数据传输方式，并且非常易于扩展以满足未来的任何需求。

新的固件仍然保留了传统的 Geschwister Schneider 协议，但可以切换到新协议。新协议会发送针对 Tx 和 Rx 数据包、回显标记、调试消息、错误报告和总线负载事件的优化结构体。

您可以在演示应用程序中测试所有这些功能，上面的屏幕截图使用了来自计算机 CPU 的时间戳。它们被转换为精度为 1 微秒的本地时间。当您设置标志 `GS_DevFlagTimestamp` 时，演示程序会切换到适配器中生成的固件时间戳。它们没有任何优势，只会产生更多的 USB 流量，并且您会看到几行“无时间戳”的行。原因是固件时间戳仅适用于从固件接收的事件。但是当您向固件发送数据包时，不会收到时间戳。

4.3 进入 DFU 模式

向接口 1 (DFU) 发送带有 `DFU_Detach` 参数的 `SETUP` 请求以启用启动模式，然后立即发送带有 `DFU_GetStatus` 参数的 `SETUP` 请求以检查执行是否成功。如果状态为 `DfuState_AppIdle`，适配器将在 300 毫秒后进入启动模式，如果状态为 `DfuState_AppDetach`，则必须向用户显示一条消息，提示其重新连接 USB 接口。仅当引脚 `BOOT0` 之前被禁用时才需要重新连接 USB 接口，演示应用程序展示了整个过程。

请参阅“Documentation”子文件夹中的 DFU 功能描述符 1.1.pdf 的第 17 页和第 22 页。(https://github.com/Elmuc/CANable-2.5-firmware-Scan-and-Candlelight/tree/main/Documentation)

第 5 部分 固件开发人员手册

这是第一个由两个不同的人编写的固件整合成一个项目的项目，新的 CANable 2.5 固件使用相同的代码库来编译 Slcan 和 Candlelight。CAN 接口、错误处理、LED 控制、进入 DFU 模式等功能都使用相同的代码，只有 USB 接口和命令执行器是单独实现的。每个项目都有一个独立的 Makefile 文件，其中包含相应的子文件夹：Slcan 或 Candlelight。

5.1 注意事项

本项目代码不适合初学者，您需要数年的嵌入式 C 编程经验和硬件知识。

5.2 添加新开发板

在 settings.h 文件中，您可以找到开发板的定义，目前包含 MKS Makerbase 和 Openlight Labs 开发板的引脚定义。新开发板必须在新 Makefile 文件中定义为 TARGET_BOARD。

5.3 添加新处理器

settings.h 文件中还包含了 ST Microelectronics 的 HAL(硬件抽象层)文件，对于其他系列的处理器，必须将所需的 HAL 文件添加到新的子文件夹中。新处理器必须在新 Makefile 文件中定义为 TARGET_MCU。

5.4 USB 接口

USB 接口是固件中最复杂的代码。意法半导体 (ST Microelectronics) 提供了一个 HAL (硬件抽象层)，它分散在 11 个文件中，文件之间相互调用。所有 USB 相关的代码都由多个中断处理程序调用。如果您不了解什么是 SETUP 请求、什么是 OUT 端点，或者阻塞端点意味着什么，我建议您阅读 Documentation 子文件夹中优秀的 USB 教程。

5.5 在 Windows 上编译 STM32 代码

您可以在网上找到很多关于如何在 Linux 上编译固件的教程，但是尽管编译过程相当简单，却没有针对 Windows 开发者的教程。

1、Cube Programmer

(原文这里是一大段对 STM32 Cube Programmer 的吐槽，这里就不放上来了)

2、安装步骤

①安装 STM32 Cube CLT(STM32 命令行编译器)，您可以从 ST 网站的下拉列表中选择最旧的版本(300 MB)。版本越新，体积越大，最新版本为 1 GB，不过您大概率不需要这么大的版本；

②安装 MinGW(适用于 Windows 的极简 GNU 框架)，此安装仅需 make.exe 和 mkdir.exe 及其 6 个 DLL 文件：libgcc_s_dw2-1.dll、libiconv-2.dll、libintl-8.dll、msys-1.0.dll、msys-iconv-2.dll、msys-intl-8.dll

③将 mkdir.exe 重命名为 mmkdir.exe，以避免与 Windows 控制台中的 MKDIR 命令冲突。

④在高级系统设置中搜索 PATH 变量，并添加两个用分号分隔的文件夹：CubeCLT 安装 arm-none-eabi-gcc.exe 文件的文件夹，MinGW 安装 make.exe 文件的文件夹，您也可以将 make.exe 和 mmkdir.exe 及其 DLL 文件复制到 CubCLT 目录，这样只需要一个文件夹即可。

⑤现在双击项目中的 Build_Slcan.cmd 文件，编译应该会开始。

此脚本会自动将编译后的 BIN 文件复制到 HUD ECU Hacker 的固件文件夹中。

⑥固件更新程序会自动将 BIN 文件转换为您可以上传的 DFU 文件。

第 6 部分 版本说明

6.1 源代码版本

25.09.14: 初始版本

25.10.23: 允许在适配器打开后修改一个过滤器。

25.11.16: 支持远程帧, 修复 Slcan 波特率 Y0 的错误, 以及在经典模式下发送 FD 数据包时报告错误的问题。

25.11.18: 修复了 S 和 Y 命令所有 Slcan 波特率的计算问题, 修复了旧版 Candlelight 数据包大小的问题。

25.11.21: C++演示程序进行了多项改进, 新增了 C#演示程序

可以看到 Elmue 仍在对 2.5 固件进行更新维护, 您可以在 GitHub 上获取最新资料。

(<https://github.com/Elmue/CANable-2.5-firmware-Slcan-and-Candlelight>)。

6.2 更新提示

1、在固件中添加新功能时, 请务必在 settings.h 文件中修改 FIRMWARE_VERSION_BCD 的值。

2、添加新的 Slcan 命令后, 必须增加 SLCAN_VERSION 的值, 并将新命令添加到文档中。



版本变更记录

[illegible]